



Military Institute of Science and Technology

Department of Computer Science and Engineering

CSE-307: Operating Systems

TERM PAPER - PART B, TRACK 1

Page Replacement under a Changing Access Pattern

A Comparison of FIFO, LRU, Optimal,
and a Simple Learned Policy

Submitted by	Fahim Azmul Hasan
Student ID	202414014
Section	A
Level and term	Level 3, Term 1
Submitted to	Lecturer Khaled Hasan Irfan
Submission date	3 October 2026

Abstract

Page replacement policies decide which page to remove when memory is full and a requested page is missing. This paper uses a Python simulation to compare FIFO, LRU, Optimal, and a simple learned policy under a changing access pattern. Each policy receives the same reference strings, allowing a fair comparison. The study examines page faults and hit ratios as repeated accesses change to random accesses or changing groups of frequently used pages. It explores how locality and the number of available frames affect replacement decisions, and whether a policy learned from one pattern remains useful when that pattern changes. The discussion connects the observed behavior to familiar memory management concepts and explains the limits of the learned approach.

1. Introduction and Background

1.1 Pages, frames, and page faults

Paging divides a process into equal-size pages and physical memory into equal-size frames. A page fits into one frame. Virtual memory allows some pages to remain outside RAM until they are needed [3]. This is the background for the present experiment.

If a requested page is already in a frame, the access is a hit. If it is absent, the access causes a page fault. When a free frame exists, the page can be loaded without replacing another page. When all available frames are occupied, the OS must choose a page to remove. FIFO, LRU, and Optimal are different ways of making that choice [1].

Term	Meaning in this paper
Reference string	The ordered list of page numbers requested by the workload.
Hit	The requested page is already in a frame.
Page fault	The requested page is not in any available frame.
Locality	A program repeatedly uses a small group of pages over a period of time.
Working set	The group of pages that a program is actively using at a particular time.

1.2 Purpose of the study

The main question is simple: what happens to page faults when the access pattern changes? A second question is whether a small learned policy can make better replacement decisions than FIFO or LRU. The experiment uses the concepts of locality, working sets, and hit and fault ratios [3, 1]. Standard memory-management texts also explain why avoiding unnecessary page faults is important [1, 3].

The program simulates page replacement on Windows using Python. It does not measure real Windows page faults, disk activity, or VMware performance.

2. Implementation of the Four Policies

2.1 FIFO, LRU, and Optimal

The simulator starts with empty frames. Each page number in the reference string is processed in order. On a hit, no page is loaded. On a fault, the page is loaded into a free frame if one exists. Otherwise, the selected policy chooses a page to replace. These steps follow the standard page replacement process [1].

Policy	Rule used when a replacement is needed
FIFO	Replace the page that entered the frames earliest. A hit does not change its arrival order.
LRU	Replace the page that has not been accessed for the longest time. Update its last-use record on every access.
Optimal	Replace the page whose next access is furthest in the future. Prefer a page that is never accessed again.

FIFO is easy to understand, but it may remove a page that is still used often. LRU uses recent access history. Optimal is included as a comparison because it has the lowest total fault count for the complete reference string. It needs future knowledge, so it cannot be used in the same way in a real running program [2]. Its future knowledge includes both halves of the string.

2.2 The simple learned policy

The Track 1 brief requires a learned component. This study uses a decision tree: a small model that learns a sequence of if-then decisions from examples [4]. It is the main addition beyond the lecture algorithms.

Each possible page to replace is described by three values: how long ago it was last used, how many times it was used in the previous 64 references, and how many times it was used since the reference string began. These values represent recent use and access frequency. The model does not receive the page's future accesses.

For training, eight separate reference strings are generated using the repeating access pattern. LRU is run on these strings to create examples of pages present when replacement is needed. The future of each training string is checked to mark which pages would be good choices under the Optimal rule. The tree learns from these examples. Its depth is limited to five decisions along any path to keep it small. None of the strings used in the final experiments is used for training.

During the experiments, the model scores the pages currently in frames. The page with the highest replacement score is removed. When scores are equal, the LRU rule is used. Access-history values continue to update, but the model is not trained again after the access pattern changes. Earlier research also studies using learned predictions to help caching decisions [5]; this paper uses only a small classroom example of that idea.

2.3 Basic correctness checks

The program was checked on short reference strings. It reproduces a standard Belady example: FIFO gives 9 faults with three frames and 10 faults with four frames [2]. Additional checks cover LRU, Optimal, repeated accesses, and the learned policy's tie rule. All six automated test methods passed.

3. Experimental Setup and Results

3.1 The same workload for every policy

Each reference string contains 6,000 accesses to 64 possible pages. The pattern changes after 3,000 accesses. Before the change, 90% of accesses follow a repeating sequence through pages 0-7; the rest access other pages. This creates a small frequently used group, similar to a program repeatedly executing a loop.

Two second-half patterns are tested. In the random pattern, every page is equally likely to be requested. In the bursty pattern, 80% of accesses use one eight-page group and 20% can use any page. The group changes every 128 accesses. This represents a program moving from one active group of pages to another.

Setting	Value
Accesses before / after the change	3,000 / 3,000
Possible page numbers	0-63
Frame counts tested	8, 12, 16, and 24
Repeated trials	Ten reference strings per second-half pattern
Main comparison in the tables	16 frames

Every policy uses exactly the same generated reference strings. The frames are not cleared when the pattern changes. This makes the comparison fair and lets the experiment show what happens when old pages remain in memory.

The hit ratio is calculated from the number of successful accesses [1]:

$$\text{Hit ratio} = (\text{Number of hits} / \text{Number of references}) \times 100\%$$

For example, 2,700 hits in 3,000 references give a 90% hit ratio and 300 faults. Hit ratio plus fault ratio equals 100%.

3.2 Measured results with 16 frames

The following tables show averages across ten trials. Average faults are rounded to whole numbers, and hit ratios to one decimal place. Exact values are available in the repository's result files. Both scenarios use the same first-half pattern.

Table 1. Before the change: repeating access pattern, 3,000 accesses per trial.

Policy	Average faults	Hit ratio (%)
FIFO	475	84.2
LRU	267	91.1
OPT	191	93.6
Learned	266	91.1

Table 2. After the change: 3,000 accesses per trial under each pattern.

Policy	Random faults	Random hit (%)	Bursty faults	Bursty hit (%)
FIFO	2255	24.8	825	72.5
LRU	2248	25.1	641	78.6
OPT	1306	56.5	421	86.0
Learned	2264	24.5	898	70.1

4. Discussion of Results

4.1 Before the change: locality helps

Before the change, pages 0-7 are used repeatedly. LRU and the learned policy both keep many of these useful pages in frames and achieve about 91% hits. FIFO gives about 84% hits because its rule considers arrival order rather than recent use. This difference is consistent with temporal locality: a page used recently is often used again soon [1].

The learned policy and LRU are almost equal before the change. Their small difference does not show a clear advantage for learning. Optimal reaches about 94% because it can check future requests before selecting a replacement.

4.2 After the change: random and bursty accesses differ

With random accesses, recent use no longer tells the policies which page will be requested next. Sixteen frames hold only one quarter of the 64 possible pages, so FIFO, LRU, and the learned policy achieve roughly 25% hits. Optimal performs better because it can see the future. This is why it is useful as a comparison, but is not a realistic replacement rule for an unknown future string.

With bursty accesses, a small working set still exists, but it moves between different groups of pages. LRU follows recent use and achieves about 79% hits. The learned policy achieves about 70%. It was trained on a pattern where the same group stays useful, so it does not follow changing groups as effectively. The old access counts can also favor pages that are no longer useful. This is a reasonable explanation of the result, rather than a separate test of each feature.

At 16 frames, the learned policy shows the largest hit-ratio drop in both tested changes. In the bursty case, its drop is about 21 percentage points, compared with about 12 for LRU. Learning from one pattern does not guarantee good decisions under a different pattern.

4.3 What happens when more frames are available?

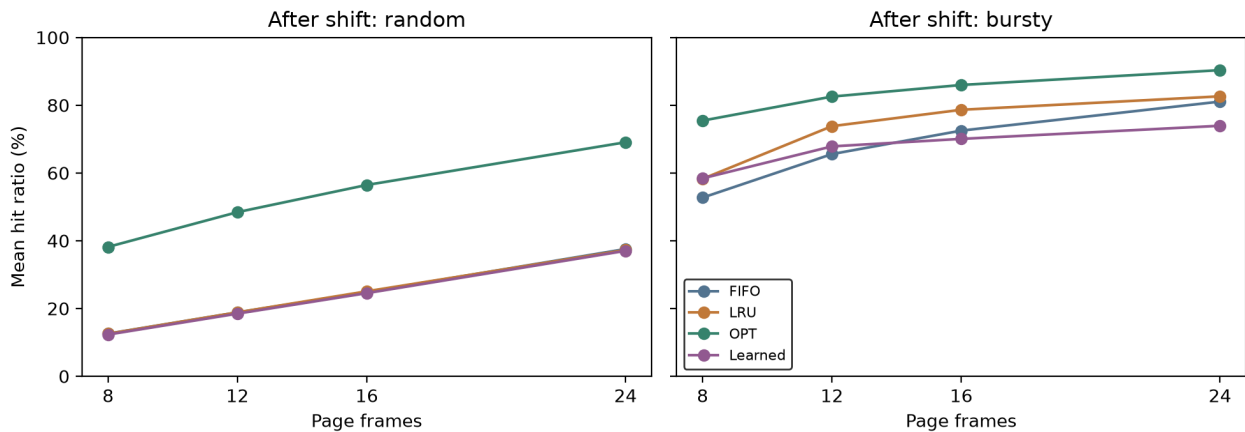


Figure 1. Average hit ratios after the change at four frame counts.

In these experiments, more frames generally allow more useful pages to remain in memory. The working-set idea explains why this helps [3]. The separate FIFO test also reminds us that more frames do not improve every possible FIFO reference string: Belady's anomaly is an exception [2].

5. Conclusion and References

5.1 Main findings and limits

This experiment connects the page-replacement algorithms to a changing reference string. The main findings are that locality helps LRU, random accesses make past history less useful, and a changing working set requires the policy to keep track of currently useful pages. The simple learned policy almost matches LRU before the change but performs worse after the change to bursts. For the main 16-frame comparison, LRU is the better simple choice after that change.

Frequent page faults can slow execution because missing pages need to be brought into memory [1, 3]. This simulation counts faults; it does not measure disk delay or effective access time. It also does not establish real system thrashing, which involves time spent servicing faults. The tested strings are generated examples, so the conclusions apply to these patterns rather than to every application.

The scripts, raw results, and figures are available in the public GitHub repository: <https://github.com/fahimeon/cse307-learned-page-replacement>. The experiment can be repeated using `python experiment.py`. AI assistance was used for implementation, running the experiments, and drafting the report. The numbers come from executed simulations. The student should review and understand the work before submission.

5.2 References

The following sources support the memory-management explanations, classical replacement policies, and simple learned component.

[1] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, Operating Systems: Three Easy Pieces, Chapter 22, "Beyond Physical Memory: Policies." <https://pages.cs.wisc.edu/remzi/OSTEP/vm-beyondphys-policy.pdf>.

[2] Cornell University, CS 4410, "Lecture 15: Page replacement," Summer 2015. <https://www.cs.cornell.edu/courses/cs4410/2015su/lectures/lec15-replacement.html>.

[3] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed., Wiley, 2018, Chapter 10, "Virtual Memory." Author's companion site: <https://www.os-book.com/OS10/>.

[4] scikit-learn developers, "Decision Trees," documentation. <https://scikit-learn.org/stable/modules/tree.html>. Accessed 3 October 2026.

[5] T. Lykouris and S. Vassilvitskii, "Competitive caching with machine learned advice," arXiv:1802.05399, 2018, revised 2020. <https://arxiv.org/abs/1802.05399>.